

01 / AUFTRAG UND SCHUTZGRENZE

Zentrale Lizenzfreigabe für EOS-Adapter

Admin **7.10.11** | Client **1.0.1** | 30.09.2026 | Protokoll v1

Integrationsstand: Admin-Dienst und gemeinsamer Client sind enthalten. Ein separater Offline-Key-Generator erzeugt neue Home-/Pro-Lizenzen. Die Einbindung der weiteren Adapter und Prüfungen am realen Gerät stehen noch aus. Dieser Leitfaden ist kein IEC-/CRA-Konformitätsnachweis.

Diese Datei beschreibt die verbindliche Einbindung der zentralen Lizenzprüfung in weitere EOS-Adapter. In diesem Repository sind der Admin-Dienst und der gemeinsame Client enthalten. Die anderen Adapter wurden mit diesem Lieferstand **noch nicht umgebaut oder am Gerät geprüft**. Ihre vorhandenen lokalen Lizenzprüfungen werden erst mit der jeweiligen Integration abgelöst.

Grundregel und Vertrauensgrenze

Der EOS Admin überprüft die vom Hersteller signierte, an die System-UUID gebundene Lizenz. Er verwaltet auch die verschlüsselte Ablage. Ein Verbraucheradapter erhält **keinen Lizenzschlüssel und keinen Entschlüsselungsschlüssel**, sondern eine kurze Freigabe für seinen eigenen Adapternamen, eine konkrete Funktion und die benötigten Mengen. Das private Signaturmaterial bleibt ausschließlich beim Hersteller/Keygen.

Ohne eine gültige aktuelle Freigabe bleiben die lizenzierten Arbeitsfunktionen gesperrt. Diagnose, Einrichtung, Lizenzstatus und die vorher geprüfte gerätespezifische Sicherheitsbehandlung müssen erreichbar bleiben. Der Admin selbst bleibt ohne Lizenz für Import, Fehlerbehebung und Systemverwaltung erreichbar; sonst wäre eine Aktivierung nach fehlendem oder abgelaufenem Schlüssel nicht möglich.

Die ioBroker-Messagebox ist eine lokale Vertrauensgrenze innerhalb des Systems. Sie ist **keine kryptografisch geschützte Gegenstelle gegenüber einem bereits kompromittierten Adapter, gemeinsamem Betriebssystembenutzer oder Root**. Solche Angreifer können möglicherweise Datenbank, Code oder Prozessspeicher ändern. Eine lokale, veränderbare JavaScript-Prüfung bietet gegen diese Angreifer keinen absoluten Kopierschutz. Die verschlüsselte Datei allein schützt auch nicht vor einem Angreifer, der zugleich auf den lokalen Ablageschlüssel oder den Prozess zugreifen kann. Für diese weitergehende Grenze sind getrennte Betriebssystemidentitäten, restriktive Schnittstellen und gegebenenfalls ein hardwaregebundener Schlüssel gesondert umzusetzen und nachzuweisen.

02 / EINBINDUNG

Client übernehmen und starten

Den vollständigen Ordner `packages/eos-license-client/` mit dessen Lizenzhinweisen aus diesem Herstellerrepository in den jeweiligen Adapter übernehmen, zum Beispiel unter `lib/eos-license-client/`. Die Dateien in den npm-Lieferumfang des Adapters aufnehmen und den tatsächlich gepackten Inhalt prüfen. Das Modul hat keine externen Runtime-Abhängigkeiten und unterstützt Node.js 22 und 24; in diesem Lieferlauf wurde Node.js 24 geprüft. Die für das jeweilige EOS-Produkt freigegebene Node-Version bleibt zusätzlich maßgeblich.

CommonJS:

```
const { createLicenseGuard, LicenseError } = require('./lib/eos-license-client');
```

ES-Modul:

```
import licenseClient from './lib/eos-license-client/index.js';  
const { createLicenseGuard, LicenseError } = licenseClient;
```

Der mitgelieferte Unterordner enthält eine eigene `package.json` mit `"type": "commonjs"`. Diesen Modulmodus beim Kopieren erhalten.

Lebenszyklus und sichere Gerätebefehle

Vor dem ersten Betriebsauftrag einen Guard erzeugen. `adapter.name` und `adapter.namespace` stammen aus der eigenen `ioBroker`-Instanz und dürfen nicht als frei wählbare Lizenzidentität konfigurierbar sein. Das folgende Muster gehört in den jeweiligen Adapter; die drei fachlichen Hilfsmethoden müssen dort passend zur Geräteanbindung implementiert und geprüft werden.

```
async function initializeLicensedOperation(adapter, configuredTotals) {  
  const guard = createLicenseGuard(adapter, {  
    adminInstance: 'eos-admin.0',  
    feature: 'energy',  
    required: {  
      chargePoints: configuredTotals.chargePoints,  
      batteries: configuredTotals.batteries,  
    },  
    onLost: async ({ code }) => {  
      // In-flight Requests/Queues abbrechen, soweit das Protokoll es erlaubt.  
      adapter.cancelPendingOperationalJobs();  
      // Nur geprüfte, begrenzte Sicherheitsaktion; keine pauschale Abschaltung.  
      await adapter.enterValidatedDeviceSafeState(code);  
    },  
  });  
  
  // Start ist bis zur ersten erfolgreichen Antwort gesperrt.  
  await guard.start();  
  return guard;  
}  
  
async function writeLicensedSetpoint(adapter, guard, requestedSetpoint) {  
  guard.assertAllowed();  
  const safeSetpoint = await adapter.validateAndLimitSetpoint(requestedSetpoint);  
  // Nach jedem await unmittelbar vor dem nächsten Betriebsbefehl erneut prüfen.  
  guard.assertAllowed();  
  return adapter.writeDeviceSetpoint(safeSetpoint);  
}  
  
async function executeLicensedBilling(adapter, guard, input) {  
  guard.assertFeatureAllowed('billing');  
  const prepared = await adapter.prepareBilling(input);  
  guard.assertFeatureAllowed('billing');  
  return adapter.commitBilling(prepared);  
}
```

03 / BETRIEBSSICHERHEIT

Sperren, beenden und prüfen

`onLost` ist verpflichtend. Er muss laufende Befehlswarteschlangen begrenzen oder abbrechen und das für den konkreten Hersteller/Firmwarestand geprüfte Verhalten bei Verlust der Betriebsfreigabe auslösen. Diese Sicherheitsaktion braucht einen eigenen, eng begrenzten Codepfad; sie darf nicht an einem bereits gesperrten `assertAllowed()` hängenbleiben. Physische Schutzfunktionen, Netzanschlussgrenzen, Watchdogs und unabhängige Gerätebegrenzungen gelten weiter. Beispielsweise ist ein pauschales Abschalten aller Speicher oder der gesamten Anlage kein allgemeingültiger sicherer Zustand.

`onLost` wird je zusammenhängender Sperrphase einmal aufgerufen. Bei einem asynchronen Callback bleibt die Freigabe gesperrt, bis er abgeschlossen ist und eine anschließende neue Prüfung erfolgreich war. Wirft der Callback einen Fehler, bleibt der Guard mit `SAFE_STATE_FAILED` gesperrt. Nach Fehlerbehebung einen neuen Guard erzeugen; ein Fehler darf nicht durch einen Neustart ohne Ursachenprüfung als erledigt gelten. Der Client protokolliert nur eine feste Diagnose ohne Fehlerinhalte oder Schlüssel.

Bei Adapter-Unload zuerst neue Arbeitsaufträge verhindern und anschließend `await guard.stop()` sowie die eigene Kommunikationsbereinigung ausführen. `stop()` sperrt sofort, verwirft verspätete Antworten und ist endgültig. Für einen neuen Lebenszyklus einen neuen Guard anlegen. Ein hängender gerätespezifischer Sicherheitscallback muss durch dessen eigene, endliche Kommunikationszeitlimits begrenzt werden.

Ein Scheduler darf weiter existieren, aber bei jedem Lauf muss er vor fachlicher Arbeit `guard.isAllowed()` prüfen. Alle tatsächlich ausführenden Pfade brauchen zusätzlich `assertAllowed()` unmittelbar vor ihrer Aktion: Gerätekommunikation, State-Änderungen mit Betriebswirkung, API-/WebSocket-Befehle, Messagebox-Befehle, Zeitpläne, Wiederholungswarteschlangen und Hintergrundaufgaben. Nur im Frontend einen Schalter auszublenden ist keine Zugriffskontrolle.

API des Guards

Methode	Verhalten
<code>start()</code>	Fragt sofort und anschließend alle 5 Sekunden; liefert zunächst <code>Promise<boolean></code> .
<code>refresh()</code>	Fragt einmal; parallele Aufrufe teilen dieselbe laufende Prüfung. Jeder tatsächlich versandte Request hat eine neue zufällige Nonce.
<code>isAllowed()</code>	Prüft synchron die Freigabe und beide Ablauffristen; bei Ablauf sofort gesperrt.
<code>assertAllowed()</code>	Wirft bei Sperre einen <code>LicenseError</code> mit Diagnosecode.
<code>isFeatureAllowed(name)</code>	Prüft die frische gemeinsame Freigabe und das Vorhandensein der konkreten Funktion; unbekannte Namen sind gesperrt.
<code>assertFeatureAllowed(name)</code>	Wirft bei fehlender Funktion <code>FEATURE_NOT_LICENSED</code> , bei allgemeiner Sperre den entsprechenden Lizenzfehler.
<code>getStatus()</code>	Liefert eine Kopie des freigegebenen Status, niemals Schlüsselmaterial.
<code>stop()</code>	Sperrt sofort und wartet auf den Sicherheitscallback; keine spätere Antwort aktiviert diesen Guard wieder.

04 / LIZENZUMFANG

Home und Pro korrekt prüfen

Nicht nur die Edition als Boolean prüfen. Entscheidend sind die ausdrücklich freigegebene Funktion, Adapterliste und Mengen in der signierten Lizenz.

Eigenschaft	Home	Pro
Ladepunkte	maximal 3, tatsächlicher lizenzierter Wert kann kleiner sein	expliziter signierter Wert; technisches Schema höchstens 1.000
Batteriesysteme	maximal 2	maximal 10
Unterstützte Funktionsnamen	energy, wallet, smartHome, microgridSlave	zusätzlich microgridMaster, multisite, billing
Freigegebene Adapter	jeweiliger Adaptername muss in der signierten Liste enthalten sein	ebenso

Ein Funktionsname im Schema bedeutet nicht, dass er automatisch in jeder Lizenz enthalten ist. Bei einer Pro-Funktion muss der Adapter diese konkrete Funktion mit `assertFeatureAllowed()` unmittelbar vor der Aktion prüfen. Ein gültiges energy-Ergebnis allein erlaubt beispielsweise keinen Microgrid-Masterbetrieb.

Pro Adapterinstanz bevorzugt **einen gemeinsamen Guard** für die benötigte Grundfunktion verwenden. Weitere Funktionen anhand derselben frischen Freigabe mit `isFeatureAllowed()` für die Anzeige und `assertFeatureAllowed()` im ausführenden Backend prüfen. Der Admin liefert hierfür die gesamte verifizierte Funktionsliste. Dafür entstehen keine weiteren Messagebox-Anfragen; die 30-Prüfungen-pro-Minute-Grenze des Dienstes wird durch den normalen 5-Sekunden-Takt nicht überschritten. Eine fehlende optionale Pro-Funktion sperrt diese Aktion, ohne vorhandene Home-Funktionen abzuschalten. Bei Verlust der gemeinsamen Freigabe sind alle Funktionen dieses Guards gesperrt.

`required` muss die tatsächlichen relevanten Mengen enthalten. Die Clientangabe ist **keine bereits implementierte systemweite Mengenaggregation**. Vor Freigabe eines Verbraucheradapters sind seine Zählweise, Mehrfachinstanzen, weitere Verbraucheradapter und die gemeinsame Anlagenkonfiguration abzugleichen, damit die Aufteilung auf mehrere Instanzen die Home-Grenzen nicht umgeht. Bei einer Konfigurationsänderung oder neuen Geräten vor der Erweiterung einen neuen Guard mit aktualisierten Anforderungen anlegen und erfolgreich prüfen. Alte Freigaben dürfen höhere Mengen nicht still übernehmen. Eigentlicher Anlagenumfang und lizenzierter Umfang dürfen nicht aus frei beschreibbaren Anzeige-States abgeleitet werden.

05 / SCHNITTSTELLE

Nachrichtenvertrag v1

Ziel ist eine explizite Instanz, standardmäßig eos-admin.0. Befehl: eos.license.check. Der Client nutzt adapter.sendTo; der Admin prüft zusätzlich den vom ioBroker-Nachrichtenbus gelieferten Absender system.adapter.NAME.N, die aktivierte Instanz, den Instanznamen und die signierte Adapterfreigabe.

```
// Anfrage: nonce ist pro Request kryptografisch zufällig, 32 kleine Hex-Zeichen.
```

```
{
  v: 1,
  nonce: requestNonce,
  adapter: adapter.name,
  feature: 'energy',
  required: { chargePoints: 3, batteries: 2 }
}

// Positive Antwort: Zeitwerte sind Unix-Epochezeit in Millisekunden.
{
  v: 1,
  nonce: requestNonce,
  valid: true,
  code: 'LICENSE_VALID',
  edition: 'home',
  features: ['energy', 'wallet', 'smartHome', 'microgridSlave'],
  limits: { chargePoints: 3, batteries: 2 },
  checkedAt: checkedAtMilliseconds,
  validUntil: maximumFifteenSecondsAfterCheck
}
```

Eine negative Antwort hat valid: false, einen Diagnosecode, edition: null, features: [] und limits: {}. Keine Freigabe aus einem Timeout oder fehlenden Feld ableiten. Keine alten Antworten, alten Keys oder lokal gespeicherten licenseValid=true-States als Ersatz verwenden. Rohkeys nicht in States, Adapterobjekte, URLs, Browser-Speicher, Backups im Repository oder Logs kopieren.

Die Messagebox-Prüfung setzt keine Freigabe für das Lesen der verschlüsselten Lizenzdatei voraus. Verbraucheradapter brauchen keinen Zugriff auf Admin-Vault, Public-Key-Provisionierung oder Keygen. Ihre normalen Geräte-, State- und Administrationsrechte müssen separat auf das notwendige Maß begrenzt werden; eine Lizenzprüfung ersetzt keine Benutzer-/Rollenberechtigung.

Zeitgrenzen und Ausfallverhalten

Der Response-Timeout beträgt höchstens 2 Sekunden; timeoutMs kann zwischen 10 und 2.000 Millisekunden verkürzt werden. Eine technische Freigabe gilt höchstens 15 Sekunden. Der Client begrenzt sie mit monotoner Laufzeit **und** der lokalen Uhrzeit. Zurückstellen der Uhr verlängert dadurch keine laufende Freigabe. Fehlende Antworten, ungültige Daten, Lizenzverlust oder überschrittene Mengen sperren beim nächsten Prüfergebnis beziehungsweise spätestens mit Ablauf der letzten Freigabe. Das ist keine harte Echtzeitusage für den Betrieb bei einem blockierten Prozess/Event-Loop. Hierfür sind Gerätewatchdogs und unabhängige Schutzfunktionen erforderlich.

06 / VERIFIKATION UND OFFENE INTEGRATION

Jeden Adapter belastbar freigeben

Vor Übergabe prüfen und mit Version, Konfiguration sowie bereinigtem Rohprotokoll belegen:

1. Ohne Lizenz/mit falscher UUID/ohne Adapterfreigabe kein Betriebsauftrag; Diagnose und Aktivierung bleiben erreichbar.
2. Home bis zur freigegebenen Menge erlaubt, jede Überschreitung gesperrt; Pro-Funktionen in Home gesperrt; Pro-Mengen ebenfalls begrenzt.
3. Admin-Neustart, gelöschte oder abgelaufene Lizenz, Timeout und Kommunikationsverlust führen innerhalb der dokumentierten Frist zur Sperre und zur sicheren Geräteaktion.
4. Replay, falsche Nonce, manipulierte/fehlende Felder, verspätete Antworten, zurückgestellte Uhr und parallele Requests erzeugen keine unberechtigte Freigabe.
5. Alle Schreibpfade einschließlich Wiederholungen und API-Befehlen prüfen die Freigabe unmittelbar vor der Aktion; Konfigurationsänderungen berücksichtigen neue Mengen.
6. Physisches Verhalten bei bereits laufendem Ladevorgang, Speicherregelung und Netzbegrenzung sowie die sichere Wiederaufnahme am konkreten Gerät testen.

Reproduzierbare Clientprüfung in diesem Repository:

```
node --test test/eos-license-client.test.cjs
```

Diese automatisierten Clienttests verwenden einen simulierten ioBroker-Transport. Sie sind keine Geräteprüfung und kein Nachweis einer unabhängigen Zertifizierung oder vollständigen IEC-/CRA-Konformität. Die Integration aller ausgelieferten Adapter und systemweite Mengenkontrolle bleiben eigene Freigabeschritte.

Separater Offline-Key-Generator

Der neue NexoWatt Offline-Key-Generator 1.0.0 wird als **separates vollständiges Paket** bereitgestellt. Er erzeugt UUID-gebundene Home- und Pro-Lizenzen im unten beschriebenen NWL2-Format. Der Admin enthält den Verifizierer und die zentrale Lizenzverwaltung; das Herstellerwerkzeug wird nicht in den Admin eingebaut. Es werden keine produktiven privaten oder öffentlichen Herstellerschlüssel vorgegeben. Das erste Schlüsselpaar entsteht erst auf dem Herstellerrechner.

Frühere NW1-/NW1E-/NW1H-Lizenzen und zeitlich begrenzte Altformate werden nicht durch ein eingebautes HMAC-Geheimnis akzeptiert. Der Hersteller muss die Berechtigung prüfen und für dieselbe UUID eine neue NWL2-Lizenz ausstellen. Umbenennen eines alten Codes reicht nicht. Es gibt keine automatische Übernahme aus `nexowatt-ui/native.licenseKey` und keinen Klartext-Fallback. Bereits verteilte gemeinsame Geheimnisse sind für neue Lizenzen nicht weiterzuverwenden.

07 / HERSTELLERWERKZEUG

Home- und Pro-Lizenzen ausstellen

Voraussetzung ist ein vertrauenswürdiger Herstellerrechner mit Node.js 22 oder 24. Der Generator hat keine externen Runtime-Pakete und benötigt zum Ausstellen keine Internetverbindung. Unter Windows `Start-Windows.cmd` öffnen; unter Linux/macOS `sh start-linux-macos.sh` ausführen. Alternativ im entpackten Generatorverzeichnis `node cli.js serve` starten. Den angezeigten privaten Sitzungslink auf **demselben Rechner** öffnen. Die Oberfläche bindet nur an 127.0.0.1; der zufällige Sitzungszugang gehört nicht in Logs, Screenshots oder Nachrichten. Dieser Lieferlauf verwendet Linux mit Node.js 24; native Windows-/macOS- und Node.js-22-Prüfungen bleiben getrennt nachzuholen.

1. Eine neue Schlüsselkennung (kid) festlegen und den Herausgebertresor mit einer starken, einmaligen Passphrase von mindestens 16 Zeichen anlegen. Passphrase sicher getrennt sichern; es gibt keine Wiederherstellungs-Hintertür.
2. Den öffentlichen Vertrauensanker als `license-trust.json` exportieren. Den angezeigten SHA-256-Fingerabdruck des öffentlichen SPKI-Schlüssels über einen getrennten, verlässlichen Übergabeweg prüfen und auf dem Zielsystem protokollieren.
3. Die tatsächliche System-UUID im EOS Admin ablesen und in den Lizenzauftrag übernehmen. Home oder Pro, erlaubte Adapter, konkrete Ladepunkt-/Speichermengen und Beginn/Ablauf prüfen. Ein leeres Ablaufdatum bedeutet unbefristet; diese Entscheidung vor dem Signieren bewusst prüfen.
4. Mit der Tresor-Passphrase signieren und die `.nwl`-Datei vertraulich übertragen. Den öffentlichen Vertrauensanker einmalig wie unten beschrieben auf dem Gerät bereitstellen, anschließend den Lizenzcode über die geschützte Admin-Lizenzseite importieren.
5. Adapterfreigaben und gesperrte Funktionen prüfen. Nach der Arbeit die lokale Sitzung beenden. Herstellertresor und Passphrase gehören weder auf Kundengeräte noch in den Admin, ein Repository, einen Supportupload oder ein unverschlüsseltes Backup.

08 / HERSTELLERSCHLÜSSEL

CLI und verschlüsselter Tresor

CLI-Alternative; die Passphrase wird jeweils verdeckt abgefragt und darf nicht als Argument oder Umgebungsvariable mitgegeben werden:

```
node cli.js init nexowatt-issuer-2026
node cli.js trust license-trust.json
node cli.js issue auftrag.json lizenz.nwl
```

Ein Auftrag enthält genau `uuid`, `edition`, `adapters`, `limits`, `notBefore` und `expiresAt`. `notBefore: null` bedeutet den Ausstellungszeitpunkt, `expiresAt: null` eine ausdrücklich unbefristete Lizenz; andere Zeitwerte sind absolute Unix-Millisekunden. Ein vollständiges Auftragsschema und Bedienhinweise liegen im separaten Generatorpaket. Ausgabedateien werden nicht still überschrieben. Mit `--vault /absoluter/pfad` lässt sich ein eigener Tresor außerhalb des Generatorverzeichnis wählen.

Standardablage: Benutzerverzeichnis `~/.nexowatt-keygen/issuer-1/issuer.nwk`. Der Ed25519-Privatschlüssel wird als PKCS#8-Material mit AES-256-GCM verschlüsselt; der Schlüssel dafür wird aus der Passphrase mit `scrypt` ($N=131072$, $r=8$, $p=1$) und zufälligem 32-Byte-Salt abgeleitet. Dazu kommen eine frische 12-Byte-Nonce, ein 16-Byte-Tag und authentifizierte Metadaten. Unter POSIX gelten 0700 für das Tresorverzeichnis und 0600 für die Datei. Unter Windows müssen die persönlichen NTFS-ACLs zusätzlich geprüft werden; POSIX-Modi bilden diese Prüfung nicht ab. Passphrasen werden nach der Anfrage aus Formularen entfernt, aber eine vollständige Löschung aller Speicherreste in JavaScript wird nicht zugesichert. Ein kompromittierter Herstellerrechner bleibt außerhalb dieser Schutzgarantie.

09 / SIGNATURVERTRAG

NWL2 und UUID-Bindung

- Verfahren ausschließlich Ed25519; privater Signaturschlüssel bleibt beim Hersteller.
- Öffentlicher Prüf-Schlüssel: PEM SubjectPublicKeyInfo, BEGIN PUBLIC KEY, Typ Ed25519.
- Token: NWL2.<payload64>.<signature64>.
- payload64: base64url ohne Padding der exakten UTF-8-JSON-Bytes.
- Signierte Bytes: ASCII NWL2. plus exakt payload64.
- Signatur: Ed25519 mit diesen Bytes, danach base64url ohne Padding. Node.js: Algorithmusparameter null.
- Der Empfänger verifiziert die Originalbytes. JSON nach der Signierung niemals umformatieren.
- Token maximal 16.384 Zeichen; kein Verifizierungsalgorithmus aus dem Token wählbar.

Das JSON hat exakt diese Felder (keine geheimen Daten, Kundennamen oder Zahlungsdaten nötig):

Feld	Vertrag
v	Zahl 2
kid	Kennung des öffentlichen Herstellerschlüssels, 1..64 ASCII-Zeichen
licenseId	Eindeutige Lizenzkennung, 1..128 ASCII-Zeichen
uuid	Die von system.meta.uuid.native.uuid gelesene UUID; trim/lowercase; normale 36-Zeichen-UUID oder ioBroker-Präfix aus 2 Buchstaben direkt davor
edition	Genau home oder pro
issuedAt	Ganze Unixzeit in Millisekunden
notBefore	Ganze Unixzeit in Millisekunden, mindestens issuedAt
expiresAt	Ganze Unixzeit in Millisekunden größer notBefore oder null für eine vom Hersteller ausdrücklich dauerhaft ausgestellte Lizenz
adapters	Eindeutige Liste erlaubter Adapter-Namen ohne iobroker.-Präfix/Instanz, z. B. nexowatt-ui, nexowatt-devices; kein Sternchen
limits	Objekt mit genau chargePoints und batteries; ganze Zahlen

Home: 1..3 Ladepunkte, 0..2 Speicher. Pro: 1..1000 Ladepunkte als technische Obergrenze des Schemas und 0..10 Speicher. Die tatsächlich verkaufte Zahl steht ausdrücklich signiert in der Lizenz. 1000 ist **keine** zugesicherte Systemleistung oder pauschale Verkaufslizenz. Home bleibt als Dauerlizenz möglich; Pro-Abrechnung und Sicherheits-Support sind getrennte Geschäftsprozesse. Zeitlizenzen verwenden ein absolutes Ablaufdatum, nicht eine durch Neuinstallation neu startende Laufzeit.

Beide Editionen: energy, wallet, smartHome, microgridSlave. Pro zusätzlich: microgridMaster, multisite, billing. Feature-Namen stammen aus dem festgelegten Servercode; kein frei eingesetztes Features-Array kann Rechte erhöhen. Die fachliche Zuordnung weiterer Apps muss beim Adapterumbau ergänzt und getestet werden. Diese begrenzte Matrix ist kein Nachweis, dass bestehende Apps bereits umgestellt sind.

10 / GERÄTEPROVISIONIERUNG

Vertrauensanker bereitstellen

Standardpfad: `/etc/nexowatt/license-trust.json`. Alternativ kann der OS-Serviceadministrator vor Prozessstart `NEXOWATT_LICENSE_TRUST_FILE` auf einen absoluten Pfad setzen. Diese Einstellung wird nicht über die Admin-Weboberfläche angeboten.

Dateiinhalt ist ein JSON-Objekt: Schlüsselkennung auf öffentlichen PEM-Text. Es wird absichtlich keine nutzbare Beispiel-Schlüsseldatei mitgeliefert. Der Hersteller exportiert den öffentlichen Teil aus dem neu eingerichteten Generator. Fingerabdruck und Übergabeweg separat prüfen; anschließend den Export durch den OS-Serviceadministrator kontrolliert bereitstellen.

Unter Linux/POSIX müssen **die Datei und alle übergeordneten Verzeichnisse einschließlich der Pfadwurzel root gehören** und dürfen nicht gruppen-/weltbeschreibbar sein, zum Beispiel `root:root` mit Datei `0644` und Verzeichnissen `0755`. Eine lediglich `root` gehörende Datei in einem austauschbaren Benutzerverzeichnis genügt nicht. Symlinks im Pfad und Mehrfach-Hardlinks auf die Datei werden abgelehnt. Ein Export im persönlichen Downloadverzeichnis ist deshalb noch kein gültig bereitgestellter Vertrauensanker. Dateirechte gezielt prüfen; keine pauschalen rekursiven Rechteänderungen auf der Anlage ausführen.

Eine Datei mit privatem Schlüssel wird abgelehnt. Unter Windows sperrt der Admin die Vertrauensankerprüfung ausdrücklich mit `TRUST_PLATFORM_UNSUPPORTED`, bis eine gleichwertige ACL-Prüfung umgesetzt und geprüft wurde. Das ist unabhängig vom separat startbaren Windows-Key-Generator. Fehlender oder ungültiger Vertrauensanker: keine Betriebsfreigabe, Admin bleibt zur Diagnose und Aktivierung erreichbar.

Schlüsselrotation: im Generator einen neuen Tresor mit neuer, eindeutiger `kid` anlegen. Den neuen öffentlichen Schlüssel zusätzlich in die vorhandene Trust-Zuordnung aufnehmen (höchstens 32 Schlüssel), Dienste neu starten, neue Lizenzen ausstellen und testen. Alten `kid` erst nach abgeschlossener Migration entfernen und erneut starten. Änderungen der Trust-Datei werden in diesem Stand beim Start gelesen; keine sofortige laufende Widerrufssynchronisation. Offline-Sperrung geschieht lokal durch Entfernen der Lizenz bzw. Entfernen des Herstellerschlüssels und Neustart. Ein zentrales sofortiges Offline-Widerrufen auf entfernten Geräten ist nicht möglich.

11 / SCHUTZ DER LIZENZDATEI

Verschlüsselte Ablage im Admin

Unter `getAbsoluteInstanceDataDir(adapter)/licensing/` liegen ausschließlich:

- `storage.key`: zufälliger lokaler AES-Schlüssel (32 Byte), kein aus der UUID abgeleitetes Geheimnis.
- `license.enc`: AES-256-GCM-verschlüsselter Lizenzcode einschließlich Zeit-Höchststand; frische 12-Byte-Nonce, 16-Byte-Tag, UUID als authentifizierte Zusatzinformation.

Verzeichnisrechte 0700, Dateien 0600, Eigentümer Laufzeitbenutzer. Keine Symlinks/Hardlinks; unsichere POSIX-Vorfahren werden abgelehnt. Insbesondere gruppenbeschreibbare ioBroker-Datenverzeichnisse müssen vorab geprüft werden. Keine pauschalen rekursiven `chmod/chown`-Befehle auf der Anlage ausführen. Die Windows-Lizenzfreigabe des Admin ist in diesem Stand durch die nicht unterstützte Vertrauensankerprüfung gesperrt; reine Node-Dateimodi würden eine Windows-ACL-Prüfung nicht ersetzen.

Die UUID ist kein Passwort. Sie bindet die Lizenz an die Installation, verhindert aber allein kein Klonen durch einen privilegierten Angreifer. AES schützt eine isoliert abgegriffene Lizenzdatei; wer Laufzeitrechte, root oder den kompletten Datenträger samt `storage.key` besitzt, kann die Lizenz entschlüsseln oder Software ändern. Höhere Schutzanforderungen benötigen getrennte OS-Benutzer, minimal berechnete Dienste, Secure Boot/Signatur-Updates und bei Bedarf TPM-gebundene Schlüssel. Keine reine JavaScript-Lösung garantiert Kopierschutz gegen den Geräteeigentümer mit root.

Der Lizenzcode ist selbst ein signiertes, lesbares Transporttoken. Er wird **nach dem Import** verschlüsselt gespeichert. Kein Klartextcode wird von Status/API an Browser oder Adapter zurückgegeben. Import nur über HTTPS oder ausschließlich lokalen Loopback-Zugriff; Browser-LocalStorage wird nicht verwendet. Keine Screenshots/Logs des Codes erstellen. Netzwerkmitschnitt-Tests und reale ioBroker-Integration stehen noch aus.

12 / BETRIEB UND ÜBERGABE

Wiederherstellung und Freigabeschritte

Lizenzdatei und storage.key gemeinsam in einem zusätzlich verschlüsselten, zugriffsgeschützten Gerätebackup sichern. Nur derselben UUID wiederherstellen. Verlust des lokalen Schlüssels bei vorhandenem Ciphertext wird nicht durch automatische Neuerzeugung verschleiert; kontrollierte Neuaktivierung durch den Hersteller erforderlich. Unvollständige oder beschädigte Sicherung bedeutet Sperre.

Davon getrennt den verschlüsselten Herstellertresor issuer.nwk sichern und die Passphrase getrennt aufbewahren. Eine Kopie des öffentlichen Vertrauensankers ersetzt den Privatschlüssel nicht. Verlust von Tresor oder Passphrase erfordert ein neues Schlüsselpaar, kontrollierte Verteilung des neuen öffentlichen Schlüssels und Neuausstellung betroffener Lizenzen; niemals einen gemeinsamen Ersatzschlüssel in den Admin einbauen.

Gültigkeitsprüfung erfolgt bei jeder Anfrage, alle fünf Sekunden zusätzlich und nach Import. Der an Verbraucher ausgegebene Zeitraum beträgt höchstens 15 Sekunden. Rückgestellte Uhr wird gegen monotone Laufzeit und verschlüsselten Zeit-Höchststand geprüft; Speicherung mindestens einmal pro Minute bei gültiger Lizenz. Vollständiges Zurückrollen des Datenträgers samt Uhr bleibt ohne Hardwareanker erkennbar unlösbar. Große legitime Zeitkorrekturen können daher einen Service-Neustart/Herstellerprüfung erfordern; kein heimlicher Freischalt-Fallback.

Eine Lizenzentfernung beendet bezahlte Betriebsfreigaben, löscht aber keine Gerätekonfiguration und stoppt nicht ungeprüft alle ioBroker-Prozesse. Der Verbraucheradapter muss Diagnose und gerätespezifische Schutzfunktionen erhalten. Anpassungen der echten Verbraucheradapter sind vor einer Produktfreigabe Pflicht.

Vor der Produktfreigabe noch offen

1. Den Herausgebertresor auf dem Herstellerrechner erzeugen, sichern und seinen öffentlichen Ed25519-Prüf Schlüssel kontrolliert auf dem Linux-Gerät bereitstellen. Home-/Pro-Lizenz für die tatsächliche UUID ausstellen und importieren.
2. Alle Verbraucheradapter mit dem gemeinsamen Guard ausstatten. Jede Betriebsaktion, optionale Pro-Funktion und Konfigurationsänderung im Backend absichern.
3. Tatsächliche Mengen systemweit abgleichen. Mehrfachinstanzen und mehrere Verbraucheradapter dürfen Home-/Pro-Grenzen nicht umgehen.
4. Ausfall, Lizenzverlust und Wiederaufnahme mit dem konkreten Hersteller, Gerät und Firmwarestand prüfen. Reales ioBroker-Verhalten, Netzwerktransport und Backup/Wiederherstellung nachweisen.
5. Automatisierte Ergebnisse und Geräteprüfungen mit Version, Konfiguration sowie bereinigten Rohbelegen zusammenführen. Erst danach eine Produktfreigabe entscheiden.

Quellen im Repository

docs/licensing/ADAPTER_INTEGRATION.md

docs/licensing/ISSUER_AND_STORAGE.md

packages/eos-license-client/

Separates Paket: NexoWatt EOS Offline-Key-Generator 1.0.0